EVIDENCIA 1: MODELO ESTRELLA DE UN DATA MART

Humberto Alexander Ramírez

Juan José Suárez Restrepo

Juan Sebastian Zapata Barrientos

TUTOR:

Antonio Jesus Valderrama Jaramillo

PREICA2601B010048

FECHA ELABORACIÓN:

05/03/2026

Modelo Estrella

La gran mayoría de organizaciones y empresas necesitan analizar grandes cantidades de datos día tras día, para evaluar y tomar decisiones. Para esto se utilizan almacenes de datos y estructuras especializadas como los Data Mart**,** que permiten organizar la información de manera eficiente para su análisis.

En este trabajo se toma la base de datos Jardinería, con el objetivo de identificar las tablas más importantes y diseñar un modelo estrella que permita analizar las ventas de la empresa. Este modelo facilita consultas analíticas como identificar el producto más vendido, la categoría con más productos y el año con mayor volumen de ventas.

- **Objetivo general**

Diseñar un **modelo estrella** utilizando la base de datos Jardinería que nos permita analizar las ventas de la empresa.

- **Objetivos específicos**
- Analizar la estructura de la base de datos Jardinería.
- Identificar las tablas relevantes para el análisis de ventas.
- Diseñar una **tabla de hechos** que represente las transacciones.
- Definir las **dimensiones** necesarias para el modelo estrella.
- Establecer las relaciones entre la tabla de hechos y las dimensiones.

En esta base de datos Jardinería se encuentra la información sobre clientes, productos, pedidos y pagos. Sin embargo, este tipo de estructura está optimizada para operaciones diarias y no para análisis.

Por esta razón se requiere diseñar un **modelo dimensional**, específicamente un **modelo estrella**, que permita realizar análisis sobre las ventas, productos y periodos de tiempo, facilitando la generación de reportes estratégicos.

Planteamiento del problema

La base de datos **Jardinería** almacena información sobre clientes, productos, pedidos y ventas mediante un modelo relacional orientado a operaciones transaccionales. Sin embargo, cuando se requiere analizar información del negocio, como identificar el producto más vendido, la categoría con más productos o el año con más ventas, la estructura actual dificulta la obtención rápida de estos resultados. Por esta razón, se plantea la necesidad de diseñar un **modelo estrella** que permita organizar la información de manera adecuada para el análisis de datos.

Análisis del problema

El modelo relacional de la base de datos Jardinería es adecuado para registrar transacciones diarias, pero no está optimizado para realizar análisis de información. Para facilitar este proceso, se propone implementar un **modelo dimensional tipo estrella**, el cual organiza los datos en una **tabla de hechos** que contiene las ventas y varias **tablas de dimensiones** que almacenan información descriptiva como productos, clientes, categorías y fechas. Este modelo permite realizar consultas analíticas de forma más sencilla y eficiente para apoyar la toma de decisiones.

Utilizamos el gestor de base de datos PostgreSQL

Iniciamos creando cada una de las tablas que tenía la base de datos jardinería, finalizamos poblando sus respectivas tablas.

```sql
CREATE TABLE oficina (
  id_oficina INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  descripcion VARCHAR(10) NOT NULL,
  ciudad VARCHAR(30) NOT NULL,
  pais VARCHAR(50) NOT NULL,
  region VARCHAR(50),
  codigo_postal VARCHAR(10) NOT NULL,
  telefono VARCHAR(20) NOT NULL,
  linea_direccion1 VARCHAR(50) NOT NULL,
  linea_direccion2 VARCHAR(50)
);


CREATE TABLE empleado (
  id_empleado INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL,
  apellido1 VARCHAR(50) NOT NULL,
  apellido2 VARCHAR(50),
  extension VARCHAR(10) NOT NULL,
  email VARCHAR(100) NOT NULL,
  id_oficina INT NOT NULL,
  id_jefe INT,
  puesto VARCHAR(50),
  FOREIGN KEY (id_oficina) REFERENCES oficina(id_oficina),
  FOREIGN KEY (id_jefe) REFERENCES empleado(id_empleado)
);


CREATE TABLE categoria_producto (
  id_categoria INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  desc_categoria VARCHAR(50) NOT NULL,
  descripcion_texto TEXT,
  descripcion_html TEXT,
  imagen VARCHAR(256)
);
```

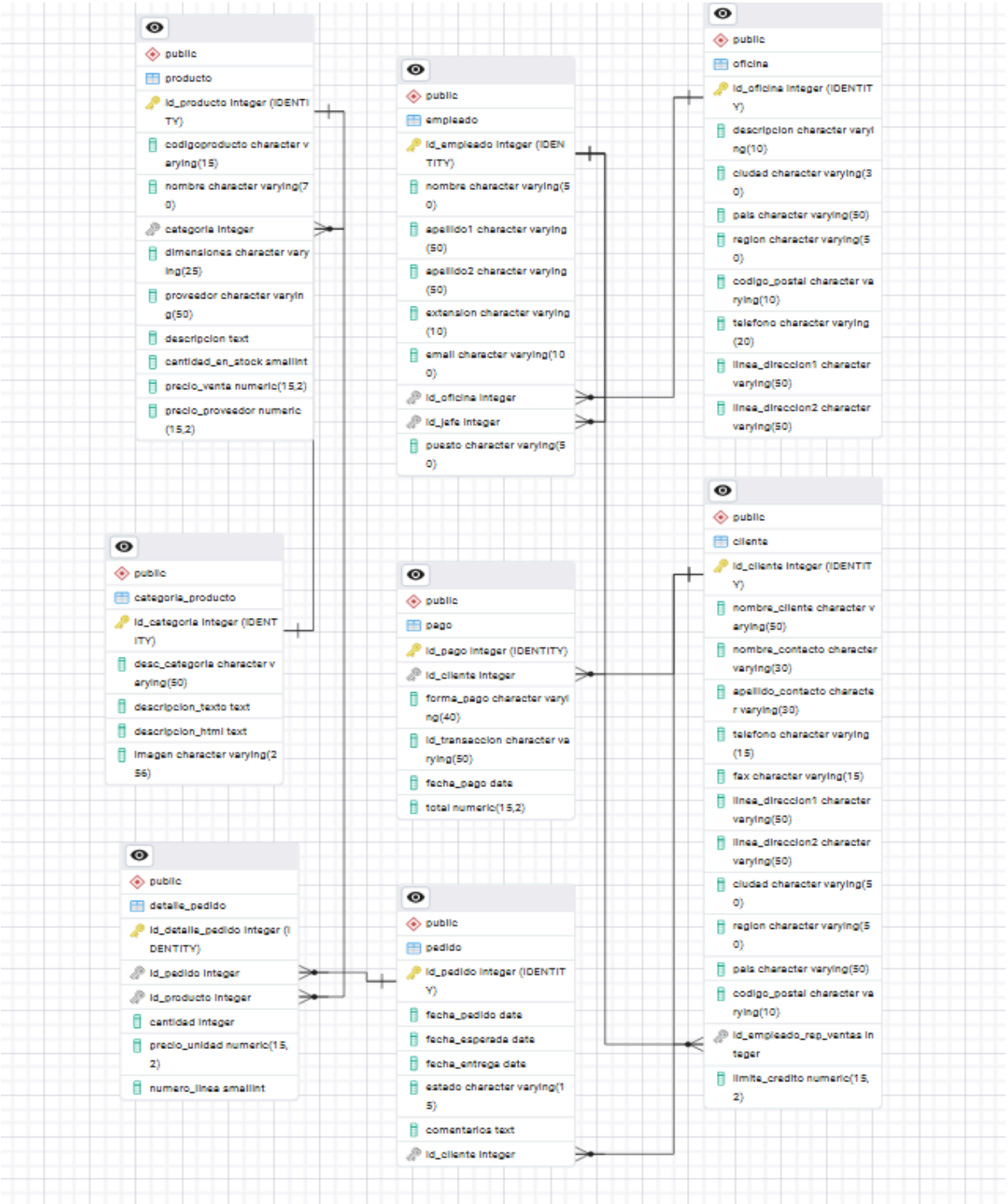
```sql
CREATE TABLE cliente (
  id_cliente INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre_cliente VARCHAR(50) NOT NULL,
  nombre_contacto VARCHAR(30),
  apellido_contacto VARCHAR(30),
  telefono VARCHAR(15) NOT NULL,
  fax VARCHAR(15) NOT NULL,
  linea_direccion1 VARCHAR(50) NOT NULL,
  linea_direccion2 VARCHAR(50),
  ciudad VARCHAR(50) NOT NULL,
  region VARCHAR(50),
  pais VARCHAR(50),
  codigo_postal VARCHAR(10),
  id_empleado_rep_ventas INT,
  limite_credito NUMERIC(15,2),
  FOREIGN KEY (id_empleado_rep_ventas) REFERENCES empleado(id_empleado)
);

CREATE TABLE pedido (
  id_pedido INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  fecha_pedido DATE NOT NULL,
  fecha_esperada DATE NOT NULL,
  fecha_entrega DATE,
  estado VARCHAR(15) NOT NULL,
  comentarios TEXT,
  id_cliente INT NOT NULL,
  FOREIGN KEY (id_cliente) REFERENCES cliente(id_cliente)
);

CREATE TABLE producto (
  id_producto INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  codigoproducto VARCHAR(15) NOT NULL,
  nombre VARCHAR(70) NOT NULL,
  categoria INT NOT NULL,
  dimensiones VARCHAR(25),
  proveedor VARCHAR(50),
  descripcion TEXT,
  cantidad_en_stock SMALLINT NOT NULL,
  precio_venta NUMERIC(15,2) NOT NULL,
  precio_proveedor NUMERIC(15,2),
  FOREIGN KEY (categoria) REFERENCES categoria_producto(id_categoria)
```

```sql
CREATE TABLE detalle_pedido (
  id_detalle_pedido INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_pedido INT NOT NULL,
  id_producto INT NOT NULL,
  cantidad INT NOT NULL,
  precio_unidad NUMERIC(15,2) NOT NULL,
  numero_linea SMALLINT NOT NULL,
  FOREIGN KEY (id_pedido) REFERENCES pedido(id_pedido),
  FOREIGN KEY (id_producto) REFERENCES producto(id_producto)
);

CREATE TABLE pago (
  id_pago INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_cliente INT NOT NULL,
  forma_pago VARCHAR(40) NOT NULL,
  id_transaccion VARCHAR(50) NOT NULL,
  fecha_pago DATE NOT NULL,
  total NUMERIC(15,2) NOT NULL,
  FOREIGN KEY (id_cliente) REFERENCES cliente(id_cliente)
);
```

```sql
INSERT INTO oficina
(descripcion, ciudad, pais, region, codigo_postal, telefono, linea_direccion1, linea_direccion2)
VALUES
('BCN-ES','Barcelona','España','Barcelona','08019','+34 93 3561182','Avenida Diagonal, 38','3A escalera Derecha'),
('BOS-USA','Boston','EEUU','MA','02108','+1 215 837 0825','1550 Court Place','Suite 102'),
('LON-UK','Londres','Inglaterra','EMEA','EC2N 1HN','+44 20 78772041','52 Old Broad Street','Ground Floor'),
('MAD-ES','Madrid','España','Madrid','28032','+34 91 7514487','Bulevar Indalecio Prieto, 32',''),
('PAR-FR','Paris','Francia','EMEA','75017','+33 14 723 4404','29 Rue Jouffroy d''abbans',''),
('SFC-USA','San Francisco','EEUU','CA','94080','+1 650 219 4782','100 Market Street','Suite 300'),
('SYD-AU','Sydney','Australia','APAC','NSW 2010','+61 2 9264 2451','5-11 Wentworth Avenue','Floor #2'),
('TAL-ES','Talavera de la Reina','España','Castilla-LaMancha','45632','+34 925 867231','Francisco Aguirre, 32','5º piso (exterior)'),
('TOK-JP','Tokyo','Japón','Chiyoda-Ku','102-8578','+81 33 224 5000','4-1 Kioicho','');
```

```sql
INSERT INTO empleado
(nombre, apellido1, apellido2, extension, email, id_oficina, id_jefe, puesto)
VALUES
('Marcos','Magaña','Perez','3897','marcos@jardineria.es',8,NULL,'Director General'),
('Ruben','López','Martinez','2899','rlopez@jardineria.es',8,1,'Subdirector Marketing'),
('Alberto','Soria','Carrasco','2837','asoria@jardineria.es',8,2,'Subdirector Ventas'),
('Maria','Solís','Jerez','2847','msolis@jardineria.es',8,2,'Secretaria'),
('Felipe','Rosas','Marquez','2844','frosas@jardineria.es',8,3,'Representante Ventas'),
('Juan Carlos','Ortiz','Serrano','2845','cortiz@jardineria.es',8,3,'Representante Ventas'),
('Carlos','Soria','Jimenez','2444','csoria@jardineria.es',4,3,'Director Oficina'),
('Mariano','López','Murcia','2442','mlopez@jardineria.es',4,7,'Representante Ventas'),
('Lucio','Campoamor','Martín','2442','lcampoamor@jardineria.es',4,7,'Representante Ventas'),
('Hilario','Rodriguez','Huertas','2444','hrodriguez@jardineria.es',4,7,'Representante Ventas'),
('Emmanuel','Magaña','Perez','2518','manu@jardineria.es',1,3,'Director Oficina'),
('José Manuel','Martinez','De la Osa','2519','jmmart@hotmail.es',1,11,'Representante Ventas'),
('David','Palma','Aceituno','2519','dpalma@jardineria.es',1,11,'Representante Ventas'),
('Oscar','Palma','Aceituno','2519','opalma@jardineria.es',1,11,'Representante Ventas'),
('Francois','Fignon','','9981','ffignon@gardening.com',5,3,'Director Oficina'),
('Lionel','Narvaez','','9982','lnarvaez@gardening.com',5,15,'Representante Ventas'),
('Laurent','Serra','','9982','lserra@gardening.com',5,15,'Representante Ventas'),
('Michael','Bolton','','7454','mbolton@gardening.com',6,3,'Director Oficina'),
('Walter Santiago','Sanchez','Lopez','7454','wssanchez@gardening.com',6,18,'Representante Ventas'),
('Hilary','Washington','','7565','hwashington@gardening.com',2,3,'Director Oficina'),
('Marcus','Paxton','','7565','mpaxton@gardening.com',2,20,'Representante Ventas'),
('Lorena','Paxton','','7665','lpaxton@gardening.com',2,20,'Representante Ventas'),
('Nei','Nishikori','','8734','nnishikori@gardening.com',9,3,'Director Oficina'),
```

Luego de crear cada una de las tablas y realizar la población de datos PostgreSQL tiene una opción que nos permite ver el funcionamiento de cada una de las tablas en un diagrama relacional llamado ERD For Data base.



Entidades del diagrama relacional

Oficina
Campos: ID_Oficina, Descripción, Ciudad, País, Región, Codigo_Postal, Teléfono, Línea_Dirección1, Línea_Dirección2**.**
Descripción: Almacena las oficinas de la empresa.
Relaciones: Empleado.ID_Oficina → Oficina.ID_Oficina

Empleado
Campos: ID_Empleado, Nombre, Apellido1, Apellido2, Extensión, Email, ID_Oficina, ID_Jefe, Puesto
Descripción: Almacena a los empleados de la empresa, su oficina, jefe y puesto.
Relaciones: Empleado.ID_Jefe → Empleado.ID_Empleado

Categoría_Producto
Campos: ID_Categoría, Desc_Categoría, Descripción_Texto, Descripción_HTML, Imagen.
Descripción: Clasifica los productos dentro de categorías específicas.
Relaciones: Producto.ID_Categoría → Categoría_Producto.ID_Categoría

Cliente
Campos: ID_Cliente, Nombre_Cliente, Nombre_Contacto, Apellido_Contacto, Teléfono, Fax, Línea_Dirección1, Línea_Dirección2, Ciudad, Región, País, Código_Postal, ID_Empleado_Rep_Ventas, Límite_Crédito.
Descripción: Almacena toda la información de los clientes.
Relaciones: Pedido.ID_Cliente → Cliente.ID_Cliente

Pedido
Campos: ID_Pedido, Fecha_Pedido, Fecha_Esperada, Fecha_Entrega, Estado, Comentarios, ID_Cliente.
Descripción: Almacena los pedidos que hacen los clientes.
Relaciones: Detalle_Pedido.ID_Pedido → Pedido.ID_Pedido

Producto
Campos: ID_Producto, CódigoProducto, Nombre, Categoría, Dimensiones, Proveedor, Descripción, Cantidad_En_Stock, Precio_Venta, Precio_Proveedor.
Descripción: Guarda la información de todos los productos que tiene la empresa.
Relaciones: Producto.ID_Categoría → Categoría_Producto.ID_Categoría

Detalle_Pedido
Campos: ID_Detalle_Pedido, ID_Pedido, ID_Producto, Cantidad, Precio_Unidad, Número_Línea.
Descripción: Guarda la información detallada de los pedidos.
Relaciones: Detalle_Pedido.ID_Producto → Prducto.ID_Producto

Pago
Campos: ID_Pago, ID:Cliente, Forma_Pago, ID_Transacción, Fecha_Pago, Total.
Descripción: Recibe la información de los pagos y la guarda.
Relaciones: Pago.ID_Cliente → Cliente.ID_Cliente

Relaciones entre tablas

Oficina (1:N) Empleado: Una oficina tiene muchos empleados y cada empleado pertenece a una oficina.

Empleado (1:N) Cliente: Un empleado tiene muchos clientes y a cada cliente se le asigna un empleado.

Empleado (N:1) Empleado(Jefe): Los empleados tienen un jefe y un jefe puede tener muchos empleados.

Cliente (1:N) Pedido: Un cliente puede tener muchos pedidos y un pedido le pertenece a un único cliente.

Pedido (1:N) Detalle_Pedido: Un Pedido puede tener muchos detalles y esos detalles le pertenecen a un solo pedido.

Producto (1:N) Detalle_Pedido: Un producto puede aparecer en muchos pedidos y cada detalle de pedido refiere un único producto.

Categoría_Producto (N:1) Producto: Una categoría puede tener muchos productos y un producto solo pertenecer a una categoría.

Cliente (1:N) Pago: Un cliente puede hacer muchos pagos y cada pago solo lo realiza un cliente.

En la base de datos se pudo identificar varias tablas importantes o más relevantes:

Cliente = información del cliente
Pedido = información de pedido
Detalle pedido = productos incluidos en cada pedido
Producto = información de producto
Categoría producto = categorías de producto
Pagos = pagos realizados

Para llevar el análisis de las ventas, la tabla que necesitamos o que en este caso es más importante es;

detalle_pedido

Porque contiene:

- producto vendido
- cantidad
- precio
- pedido asociado

Esta sería nuestra tabla de hechos.

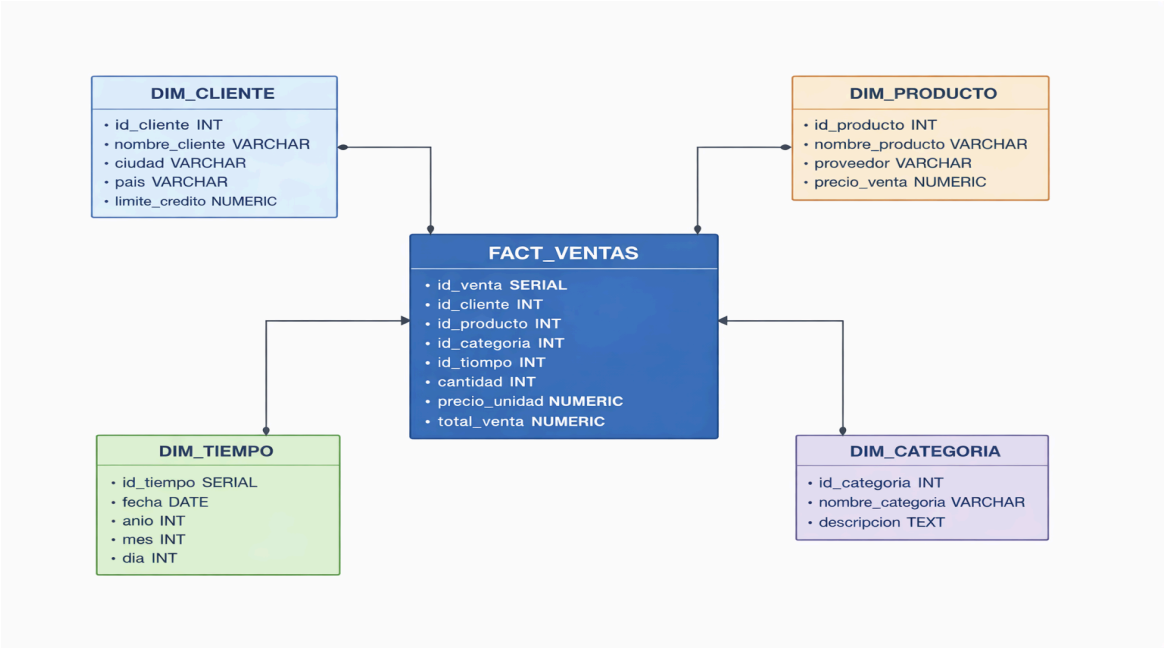
Para construir el modelo estrella, primero revisamos la base de datos Jardinería. Buscamos las transacciones y los datos característicos.

Tabla de hechos: Se define a partir de la tabla que contiene las ventas o transacciones. En este caso, usamos detalle_pedido, porque en esta están los productos vendidos, la cantidad y el precio. Esta tabla guarda las medidas numéricas que se buscan analizar.

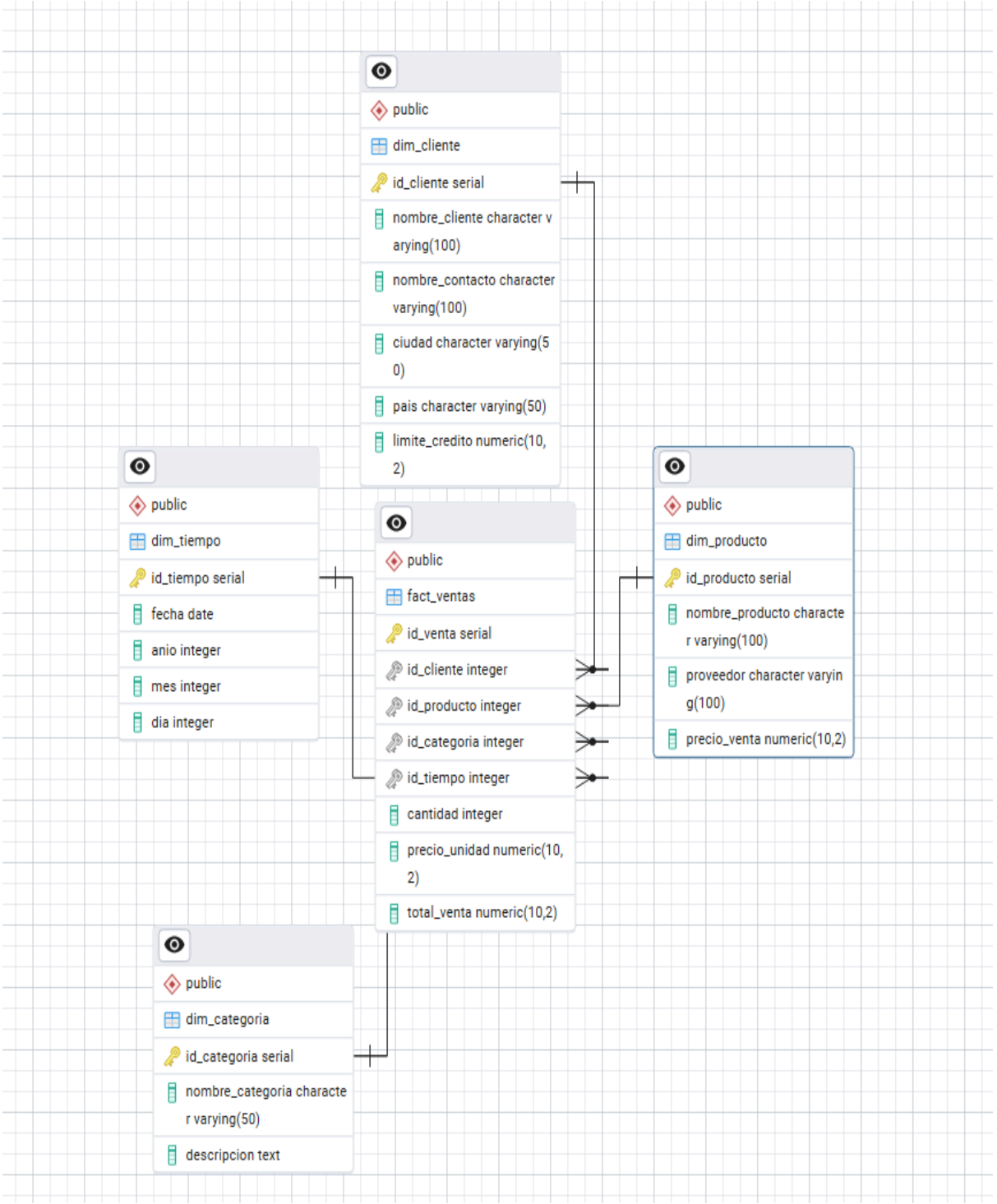
Tablas de dimensiones: Se crean con las tablas que describen la información de la venta. Por ejemplo: cliente → información del cliente producto → información del producto categoria_producto → tipo de producto pedido/fecha → fecha de la venta Las dimensiones dan contexto a los datos de la tabla de hechos. Permiten analizar las ventas por cliente, producto, categoría o fecha. Esto forma la estructura del modelo estrella.

```
#dimension cliente

CREATE TABLE dim_cliente (
    id_cliente SERIAL PRIMARY KEY,
    nombre_cliente VARCHAR(100),
    nombre_contacto VARCHAR(100),
    ciudad VARCHAR(50),
    pais VARCHAR(50),
    limite_credito NUMERIC(10,2)
);



#dimension producto

CREATE TABLE dim_producto (
    id_producto SERIAL PRIMARY KEY,
    nombre_producto VARCHAR(100),
    proveedor VARCHAR(100),
    precio_venta NUMERIC(10,2)
);


#dimension categoria

CREATE TABLE dim_categoria (
    id_categoria SERIAL PRIMARY KEY,
    nombre_categoria VARCHAR(50),
    descripcion TEXT
);



#dimension tiempo

CREATE TABLE dim_tiempo (
    id_tiempo SERIAL PRIMARY KEY,
    fecha DATE,
    anio INT,
    mes INT,
    dia INT
```

```sql
#tabla de hechos
CREATE TABLE fact_ventas (
    id_venta SERIAL PRIMARY KEY,
    id_cliente INT,
    id_producto INT,
    id_categoria INT,
    id_tiempo INT,
    cantidad INT,
    precio_unidad NUMERIC(10,2),
    total_venta NUMERIC(10,2),

    CONSTRAINT fk_cliente
        FOREIGN KEY (id_cliente) REFERENCES dim_cliente(id_cliente),

    CONSTRAINT fk_producto
        FOREIGN KEY (id_producto) REFERENCES dim_producto(id_producto),

    CONSTRAINT fk_categoria
        FOREIGN KEY (id_categoria) REFERENCES dim_categoria(id_categoria),

    CONSTRAINT fk_tiempo
        FOREIGN KEY (id_tiempo) REFERENCES dim_tiempo(id_tiempo)
);
```



Nuestro modelo estrella lo realizamos de dos maneras, uno manual y el otro es el de nuestro gestor de base de datos PostgreSQL



Entidades en el diagrama de estrella

Fact_Ventas
Campos: ID_Venta, ID_Cliente, ID_Producto, ID_Categoría, ID_Tiempo, Cantidad, Precio_Unidad, Total_Venta.
Descripción: Esta es la tabla de hechos, almacena las facturas de venta.

Dim_Producto
Campos: ID_Producto, Nombre_Producto, Proveedor, Precio_Venta.
Descripción: Contiene la información de los productos y sus proveedores.

Dim_Categoría
Campos: ID_Categoría, Nombre_Categoría, Descripción.
Descripción: Define las categorías de los productos.

Dim_Tiempo
Campos: ID_Tiempo, Fecha, Año, Mes, Día.
Descripción: Esta tabla guarda los tiempos tanto en días, como en meses o años.

Dim_Cliente
Campos: ID_Cliente, Nombre_Cliente, Nombre_Contacto, Ciudad, País, Límite_Crédito.
Descripción: Contiene la información de los clientes y capacidad de crédito.

Relaciones entre tablas

Cliente (1:N) Ventas: Un cliente puede tener muchas ventas y una venta le pertenece a un solo cliente.

Producto (1:N) Ventas: Un producto puede aparecer en muchas ventas y cada registro refiere un producto.

Categoría (1:N) Ventas: Una categoría agrupa muchos productos y cada venta asocia una categoría.

Tiempo (1:N) Ventas: Una fecha puede tener muchas ventas y cada venta se registra en una sola fecha.

Conclusiones:

El modelo estrella diseñado a partir de la base de datos Jardinería permite organizar la información de ventas de manera clara y estructurada. Al separar los datos en una tabla de hechos y tablas de dimensiones como producto, cliente, categoría y tiempo, se facilita el análisis de la información. Gracias a este modelo es posible identificar rápidamente el producto más vendido, la categoría con más productos y el año con mayores ventas. Por lo tanto, el modelo estrella mejora el análisis de datos y apoya la toma de decisiones dentro de la empresa